

Série F : Récursivité – SOLUTIONS -**Exercice F.1:**

a) Pour N positif ou zéro:

$$\begin{cases} X^0 = 1 \\ X^N = X^{N-1} \cdot X & \text{pour } N > 0 \end{cases}$$

```
public static double power(double x, int n)
{
    if (n==0) return 1;
    else      return power(x,n-1)*x;
}
```

b) Pour N positif, négatif ou zéro:

$$\begin{cases} X^0 = 1 \\ X^N = X^{N-1} \cdot X & \text{pour } N > 0 \\ X^N = 1/X^{-N} & \text{pour } N < 0 \end{cases}$$

```
public static double power(double x, int n)
{
    if (n==0)      return 1;
    else if (n>0)  return power(x,n-1)*x;
    else /* n<0*/  return 1/power(x,-n);
}
```

Exercice F.2:

GCD : Greatest Common Divisor

PGCD : Plus grand commun diviseur

GGT : Grösster gemeinsamer Teiler

$$\begin{cases} \text{gcd}(x, 0) = x \\ \text{gcd}(x, y) = \text{gcd}(y, x \% y) & \text{pour } y > 0 \end{cases}$$

Explications voir : http://de.wikipedia.org/wiki/Euklidischer_Algorithmus

```
public static double gcd(int x, int y)
{
    if (y==0) return x;
    else      return gcd(y,x%y);
}
```

Exercice F.3:

```
public boolean machin(int x)
{
    if (x==0) return true;
    else      return !machin(x-1);
}
```

a) La suite des appels récursifs et le résultat pour l'appel **machin(3)** :

```
machin(3)
= ! machin(2)
= !! machin(1)
= !!! machin(0)
= !!! true
= false
```

b) La méthode **machin** sert à déterminer la parité d'un nombre entier positif
(**true** <=> pair, **false** <=> impair)

c) La méthode **machin** retourne un résultat pour $X \geq 0$.

```
public double truc( double x, int y)
{
    if      (y==0)    return 1;
    else if (y%2==0) return truc(Math.pow(x,2),y/2);
    else            return truc(x,y-1) * x;
}
```

d) La suite des appels récursifs et le résultat pour l'appel **truc(2,5)** :

```
truc(2,5)
= truc(2,4) * 2
= truc(4,2) * 2
= truc(16,1) * 2
= truc(16,0) * 2 * 16
= 1 * 2 * 16
= 32
```

e) La méthode **truc(X,Y)** sert à calculer X^Y pour un réel X et un entier positif Y.

Exercice F.4: nombres de *Fibonacci*

a) la méthode récursive **fibo_r**

```
public static long fibonacci(int n) {
    if (n<=2) return 1;
    else return fibonacci(n-1) + fibonacci(n-2);
}
```

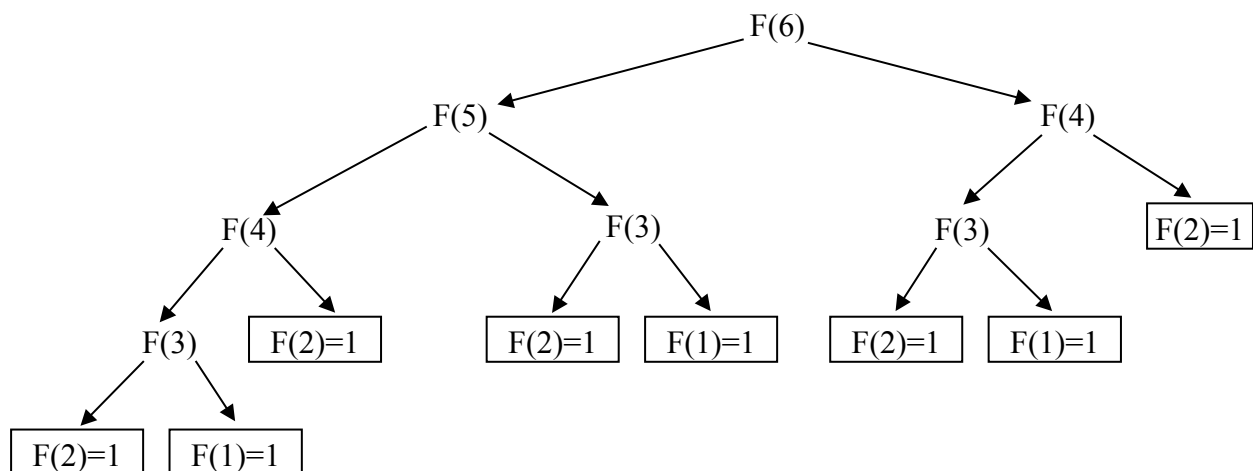
b) la méthode itérative **fibo_i**

```
public static long fibonacci(int n) {
    if (n<=2) return 1;
    else {
        long xOld    = 1;
        long xOlder  = 1;
        long xNew    = 2;
        for (int i=3 ; i<=n ; i++) {
            xNew      = xOld + xOlder;
            xOlder    = xOld;
            xOld       = xNew;
        }
        return xNew;
    }
}
```

c) Conclusions de la comparaison de **fibo_r** et **fibo_i**:

La version récursive de l'algorithme est beaucoup plus lente que la version itérative. Ceci est dû au fait que pour chaque unité, le nombre d'appels récursifs est approximativement doublé! (→ voir aussi (d)-(f)).

d) Représentation graphique (arbre) de la suite des appels récursifs pour **fibor(6)**!



e)

nar(1) =	0	fibonacci(1) =	1
nar(2) =	0	fibonacci(2) =	1
nar(3) =	2	fibonacci(3) =	2
nar(4) =	4	fibonacci(4) =	3
nar(5) =	8	fibonacci(5) =	5
nar(6) =	14	fibonacci(6) =	8
nar(7) =	24	fibonacci(7) =	13
nar(8) =	40	fibonacci(8) =	21
nar(9) =	66	fibonacci(9) =	34
nar(10) =	108	fibonacci(10) =	55

```

nar(1) = nar(2) = 0
nar(N) = nar(N-1) + nar(N-2) + 2
➔ nar(N) = ( fibo(N) - 1 ) * 2

```

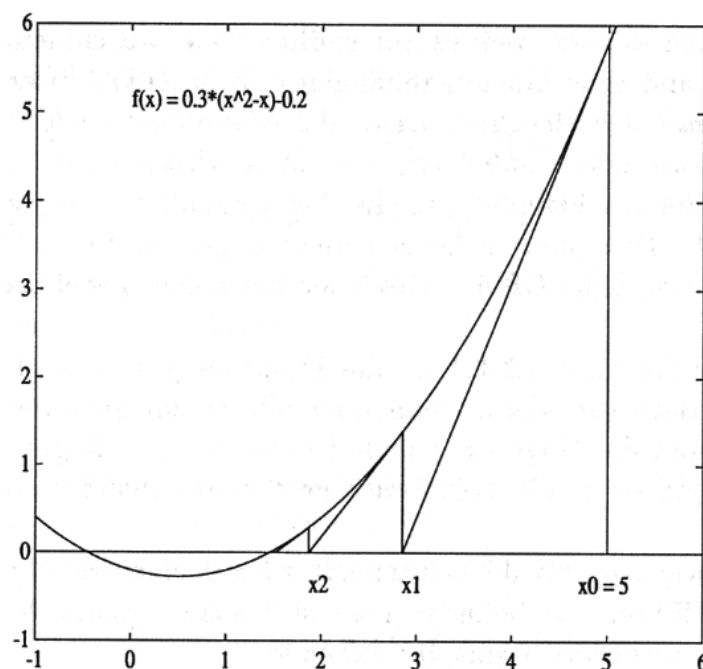
Exercice F.5: racine carrée par la formule de Héron
(application du procédé de Newton)

$$\begin{cases} u_0 = a \\ u_{n+1} = \frac{1}{2} \left(u_n + \frac{a}{u_n} \right) \end{cases}$$

Satz 3.2 (Newtonverfahren) Verwendet man zur Berechnung der einfachen Nullstelle s von f die Iterationsform

$$x = F(x) = x - \frac{f(x)}{f'(x)}, \quad (3.31)$$

dann konvergiert die Iterationsfolge $\{x_k\}$ bei geeignetem Startwert quadratisch gegen s .



Beispiel 3.1 Wir wollen einen Algorithmus zur Berechnung der Quadratwurzel entwickeln. $x = \sqrt{a}$ ist Lösung der Gleichung

$$f(x) = x^2 - a = 0.$$

Es ist

$$x - \frac{f(x)}{f'(x)} = x - \frac{x^2 - a}{2x} = (x + a/x)/2.$$

Wir erhalten somit die Iterationsformel von Heron

$$x_{k+1} = (x_k + a/x_k)/2, \quad (3.32)$$

welche eine quadratisch gegen $\sqrt{a}$ konvergente Folge $\{x_k\}$ liefert.

```
public static double newton( double a, int n )
{
    if (n==0) return a;
    else
    {
        double res = newton(a,n-1);
        return 1.0/2 * (res + a/res);
    }
}
```

Attention :

Il est important d'utiliser une variable pour ne pas provoquer deux appels récursifs à chaque étape. Le temps d'exécution évoluerait exponentiellement avec n.

Exemple : Pour $n=30$ on produirait alors 2^{30} , c.-à-d. plus que 1 milliard d'appels récursifs au lieu de 30 appels récursifs.

Exercice F.6: Séries de Taylor

Les développements en série de Taylor des fonctions *exp*, *cosinus* et *sinus* de la variable réelle *x* peuvent s'écrire :

$$e^x = \frac{x^0}{0!} + \frac{x^1}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots = \sum_{n=0}^{\infty} \left(\frac{x^{4n}}{(4n)!} - \frac{x^{4n+2}}{(4n+2)!} \right)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots = \sum_{n=0}^{\infty} \left(\frac{x^{4n+1}}{(4n+1)!} - \frac{x^{4n+3}}{(4n+3)!} \right)$$

Pour chacune des séries ci-dessus, écrire une méthode récursive qui effectue le développement de la série jusqu'au terme **n** (pour un réel **x** et un entier positif **n** donnés).

```
public static double exp(double x, int n)
{
    if (n==0)    return 1;
    else        return exp(x,n-1) + power(x,n)/factorial(n);
}

public static double cos(double x, int n)
{
    if (n==0)    return 1 - power(x,2)/factorial(2);
    else        return cos(x,n-1) + power(x,4*n) / factorial(4*n) -
                        power(x,4*n+2) / factorial(4*n+2);
}

public static double sin(double x, int n)
{
    if (n==0)    return x - power(x,3)/factorial(3);
    else        return sin(x,n-1) + power(x,4*n+1) / factorial(4*n+1) -
                        power(x,4*n+3) / factorial(4*n+3);
}
```

Comme les valeurs pour N doivent être positives, on peut aussi envisager la solution suivante pour éviter les calculs explicites pour les conditions d'arrêt:

```
public static double cos2(double x, int n)
{
    if (n==-1) return 0;
    else        return cos2(x,n-1) + power(x,4*n) / factorial(4*n) -
                        power(x,4*n+2) / factorial(4*n+2);
}

public static double sin2(double x, int n)
{
    if (n==-1) return 0;
    else        return sin2(x,n-1) + power(x,4*n+1) / factorial(4*n+1) -
                        power(x,4*n+3) / factorial(4*n+3);
}
```

Exercice F.7:

Pour les cas suivants, éliminez la boucle en effectuant les mêmes opérations à l'aide d'un sous-programme récursif! Indiquez aussi les parties qui font appel aux sous-programmes récursifs!

a)

```
for (int i=1 ; i<=10 ; i++)  
    System.out.println(i);
```

```
public static void recuSimpleUp(int i)  
{  
    if (i<=10)  
    {  
        System.out.println(i);  
        recuSimpleUp(i+1);  
    }  
}
```

→ Appel initial : **recuSimpleUp**(1);

Solution alternative:

```
public static void recuSimpleDown(int i)  
{  
    if (i>=1)  
    {  
        recuSimpleDown(i-1);  
        System.out.println(i);  
    }  
}
```

→ Appel initial : **recuSimpleDown**(10);

**Attention : le bloc d'instructions (...println...) se trouve DERRIERE l'appel récursif !
==> les nombres sont bien imprimés dans l'ordre ascendant 1,2,3,...,10**

En général:

```
for (int i=1 ; i<=n ; i++)  
    <bloc d'instruction>
```

```
public static void recuSimpleUp(int i, int n)  
{  
    if (i<=n)  
    {  
        <bloc d'instructions>  
        recuSimpleUp(i+1,n);  
    }  
}
```

→ Appel initial : **recuSimpleUp**(1,n);

Solution alternative:

identique à **recuSimpleDown**, mais appel initial par **recuSimpleDown**(n);

b)

```

for (int i=0 ; i<=3 ; i++)
    for (int j=0 ; j<=9 ; j++)
        System.out.println(i * 10 + j);

```

```

public static void recuDouble(int i, int j)
{
    if (i<=3) {
        if (j<=9) {
            System.out.println(i*10+j);
            recuDouble(i, j+1);
        }
        else recuDouble(i+1, 0);
    }
}

```

→ Appel initial : `recuDoubleUp(0,0);`

En général:

```

for (int i=0 ; i<=n ; i++)
    for (int j=0 ; j<=m ; j++)
        <bloc d'instruction>

```

```

public static void recuDouble(int i, int j)
{
    if (i<=n) {
        if (j<=m) {
            <Instruction bloc>;
            recuDouble(i, j+1);
        }
        else recuDouble(i+1, 0);
    }
}

```

→ Appel initial : `recuDouble(0,0);`

ou aussi:

```

public static void recuDouble(int i, int j, int n, int m)
{
    if (i<=n) {
        if (j<=m) {
            <Instruction bloc>;
            recuDouble(i, j+1, n, m);
        }
        else recuDouble(i+1, 0, n, m);
    }
}

```

→ Appel initial : `recuDouble(0,0, n,m);`

Evidemment, ici aussi, il est possible de réaliser une solution alternative comptant à rebours, mais il faut alors trouver une solution élégante pour la réinitialisation du compteur j lors du 2e appel récursif.

Exercise F.8:

Selection sort (iterative) / Tri par sélection :

```
public void selSort() {
    int pmin;
    for (int i = 0; i < tab.length - 1; i++) {
        pmin = i;
        for (int j = i + 1; j < tab.length; j++) {
            if (tab[j] < tab[pmin]) {
                pmin = j;
            }
        }
        if (i != pmin) {
            int help = tab[pmin];
            tab[pmin] = tab[i];
            tab[i] = help;
        }
    }
}
```

Linear search :

```
public int linearSearch_iterative(int n)
{
    int res = -1;
    int i = 0;
    while (i < tab.length && res == -1) {
        if (tab[i] == n) {
            res = i;
        }
        i++;
    }
    return res;
}
```

```
public int linearSearch_recursive(int n, int pos)
{
    if (pos >= tab.length)    return -1;
    else if (tab[pos] == n)    return pos;
    else /* tab[pos] != n */  return linearSearch_recursive(n, pos + 1);
}
```

→ Appel initial par : `linearSearch_recursive(n, 0)` ;

Binary search (recursive) :

```
public int binarySearch(int n, int left, int right) {
    int mid = (left + right) / 2;
    if (tab[mid] == n)        return mid;
    else if (left > right)    return -1;
    else if (tab[mid] < n)    return binarySearch(n, mid+1, right);
    else /* tab[mid] > n */  return binarySearch(n, left, mid-1);
}
```

→ Appel initial par : `binarySearch(n, 0, tab.length-1)` ;

Exercice F.9: - Quicksort -

```
private int partition(int left, int right) {
    int i = left;
    int j = right - 1;
    int pivot = tab[right];
    while (i < j) {
        while (i < right && tab[i] <= pivot)    i++;
        while (j > left && tab[j] >= pivot)    j--;
        if (i < j) {
            int help = tab[i];
            tab[i] = tab[j];
            tab[j] = help;
        }
    }
    if (tab[i] > pivot) {
        tab[right] = tab[i];
        tab[i] = pivot;
    }
    return i;
}

public void qs(int left, int right) {
    if (left < right) {
        int p = partition(left, right);
        qs(left, p-1);
        qs(p+1, right);
    }
}
```

→ Appel initial par : `qs(0, tab.length-1)`

Exercice F.10: Les tours de Hanoï

Au lieu d'essayer de trouver la suite des déplacements des disques un par un (ce qui est vite infaisable), essayons de résoudre le problème avec la réflexion standard de la pensée récursive : **Supposons le problème résolu pour déplacer $n-1$ rondelles** du piquet i au piquet j .

Alors, il y a une solution très facile pour transférer n rondelles de i en j :

- 1- on amène les $n-1$ rondelles du haut de i sur le troisième piquet $k=6-i-j$,
- 2- on prend la grande rondelle en bas de i et on la met toute seule en j ,
- 3- on amène les $n-1$ rondelles de k en j .

Ceci s'écrit :

```
public static void hanoi(int n, int i, int j)
{
    if (n>0)
    {
        hanoi(n-1, i, 6-(i+j));
        System.out.println(i+" --> "+j);
        hanoi(n-1, 6-(i+j), j);
    }
}
```

Hypothèse (trouvée de façon expérimentale):

pour $n>0$ le nombre de déplacements est 2^n-1

Démonstration par récurrence:

- $n=1$, D'après l'hypothèse, il faut effectuer $2^1-1 = 2-1 = 1$ déplacement. En effet, l'hypothèse est vérifiée pour $n=1$, car pour $n=1$ il y a 1 seul déplacement à réaliser.
- $n-1 \rightarrow n$

Supposant que notre hypothèse est vraie pour $n-1$, nous démontrons qu'elle reste vraie pour n . Référons-nous à l'algorithme:

<code>hanoi(n-1, i, 6-(i+j));</code>	$2^{n-1}-1$ dépl. (hypothèse)
<code>System.out.println(i+" --> "+j);</code>	1 dépl.
<code>hanoi(n-1, 6-(i+j), j);</code>	$2^{n-1}-1$ dépl. (hypothèse)

Donc on a pour $n>1$, le nombre de déplacements est égal à

$$\begin{aligned}
 & (2^{n-1}-1) + 1 + (2^{n-1}-1) \\
 &= 2*(2^{n-1}-1) + 1 \\
 &= (2^n - 2) + 1 \\
 &= 2^n - 1 \quad \text{q.e.d.}
 \end{aligned}$$

Pour $n=64$, les moines devraient effectuer au moins $2^{64} - 1$ déplacements et ils nécessiteraient ainsi au moins $2^{64}-1$ secondes pour accomplir la tâche. Nous sommes donc encore en sécurité pour à peu près 585 000 000 000 années...

Exercice F.11: Arbre fractal - *Fractal tree* - Baum-Fraktal

```
import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.geom.Point2D;

public class FractalTree {

    private double length = 110;
    private double f1 = 0.75;
    private double f2 = 0.75;
    private double diffAngle = 40;

    public double getLength() { return length; }
    public double getF1() { return f1; }
    public double getF2() { return f2; }
    public double getDiffAngle() { return diffAngle; }

    public void setLength(double length) { this.length = length; }
    public void setF1(double f1) { this.f1 = f1; }
    public void setF2(double f2) { this.f2 = f2; }
    public void setDiffAngle(double diffAngle) { this.diffAngle = diffAngle; }

    private Point2D drawVector(Graphics2D g, Point2D p, double length, double angle) {
        //Coloriser
        if (length>10) g.setColor(new Color(75, 50, 25));
        else if (length>5) g.setColor(Color.RED);
        else if (length>3) g.setColor(Color.YELLOW);
        else if (length>1.5) g.setColor(Color.GREEN);
        else g.setColor(new Color(0,100,0));
        //Donner une épaisseur aux branches
        g.setStroke(new BasicStroke((float)length/10+1)); // trunc(Long/10)+1;
        //Dessiner le vecteur
        Point2D pEnd=new Point2D.Float();
        pEnd.setLocation( p.getX() + length*Math.cos(Math.toRadians(angle)),
                        p.getY() + length*Math.sin(Math.toRadians(angle)));
        g.drawLine( (int)Math.round(p.getX()), (int)Math.round(p.getY()),
                   (int)Math.round(pEnd.getX()), (int)Math.round(pEnd.getY()));
        return pEnd;
    }

    private void drawFractalTree(Graphics2D g, Point2D p, double length, double angle)
    {
        if (length>1) {
            p = drawVector(g, p, length, angle);
            drawFractalTree(g, p, length*f1, angle-diffAngle);
            drawFractalTree(g, p, length*f2, angle+diffAngle);
        }
    }

    public void draw(Graphics g, Point2D p) {
        drawFractalTree((Graphics2D)g, p, length, 270);
    }
}
```